

Q1 왜 그림으로 보나

그래프는 꾸미기가 아니라, 데이터의 **말을** 알아듣는 도구입니다.

Q2 어떤 그래프를 언제 쓰나

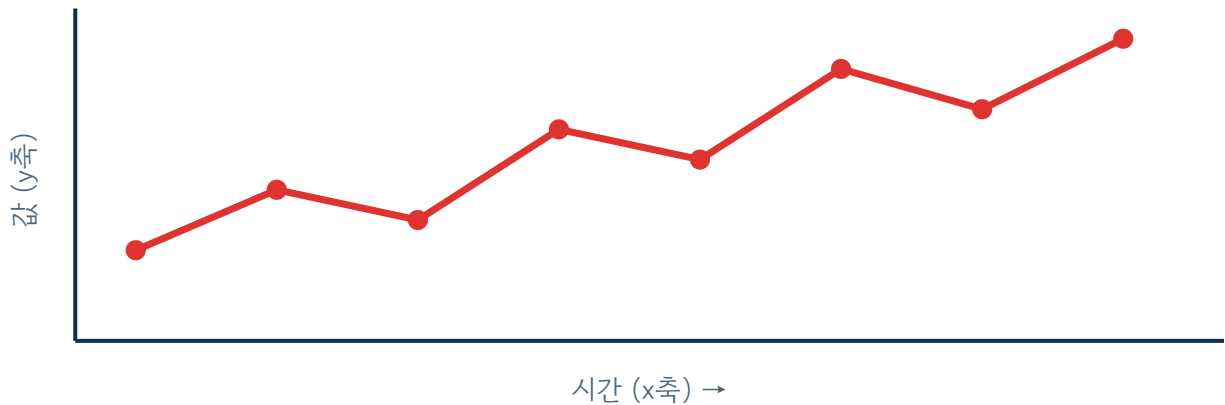
제일 중요한 건 "무엇을 보여줄까"입니다. 목적이 정해지면 그래프는 거의 자동으로 골라져요.

보고 싶은 것	어울리는 그래프
시간·순서에 따른 변화	선 그래프 (plot)
한 값이 어떻게 퍼져 있나(분포)	히스토그램 (hist)
두 값 사이의 관계	산점도 (scatter)
그룹별 요약 · 이상치	박스플롯 (boxplot)
여러 변수 관계를 한눈에	히트맵 · 페어플롯

그래프는 예쁘게가 목적이 아닙니다. "시간 변화 → 선, 분포 → 히스토그램, 관계 → 산점도"처럼 목적에 맞는 그래프를 고르는 게 시각화의 절반이에요.

Q3 선 그래프 — 시간에 따른 변화

순서(특히 시간)에 따라 값이 어떻게 변하는지 볼 때 씁니다. 주가, 기온, 월별 매출처럼 "죽 이어지는" 데이터에 딱이에요.



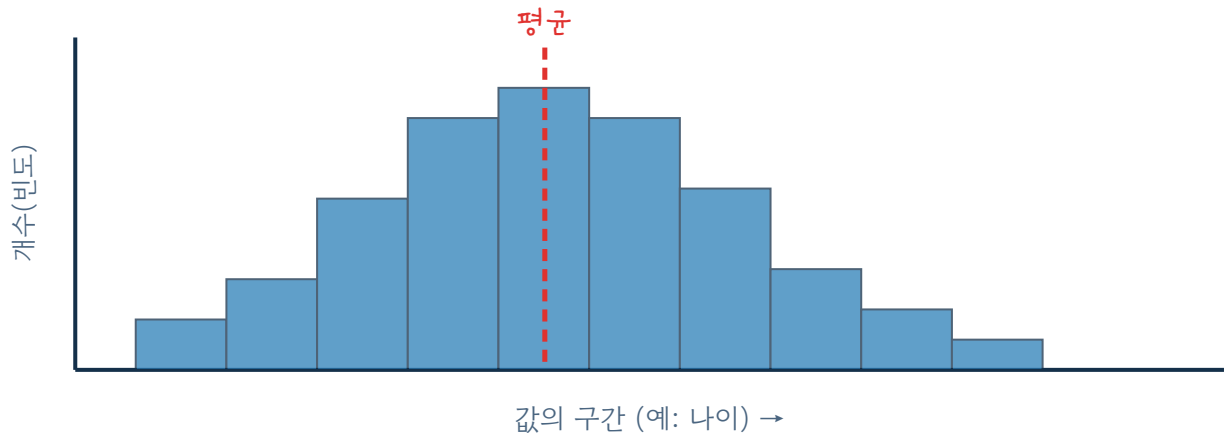
점을 선으로 이어, 오르내리는 흐름을 보여줍니다.

```
import matplotlib.pyplot as plt

plt.plot(x, y, color='orange') # 선 그래프
plt.title('Monthly sales')    # 제목
plt.xlabel('month')           # x축 이름
plt.ylabel('sales')           # y축 이름
plt.grid(True)                # 눈금선
plt.show()
```

Q4 히스토그램 — 한 값의 분포

값 하나가 어떻게 퍼져 있는지 볼 때 씁니다. "사람들 나이가 주로 몇 살대에 몰려 있나?" 같은 질문이죠. 값의 범위를 구간(bin)으로 나누고, 각 구간에 몇 개가 들어가는 막대 높이로 그립니다.



가운데가 높으면 "그 값 근처에 많이 몰려 있다"는 뜻이에요.

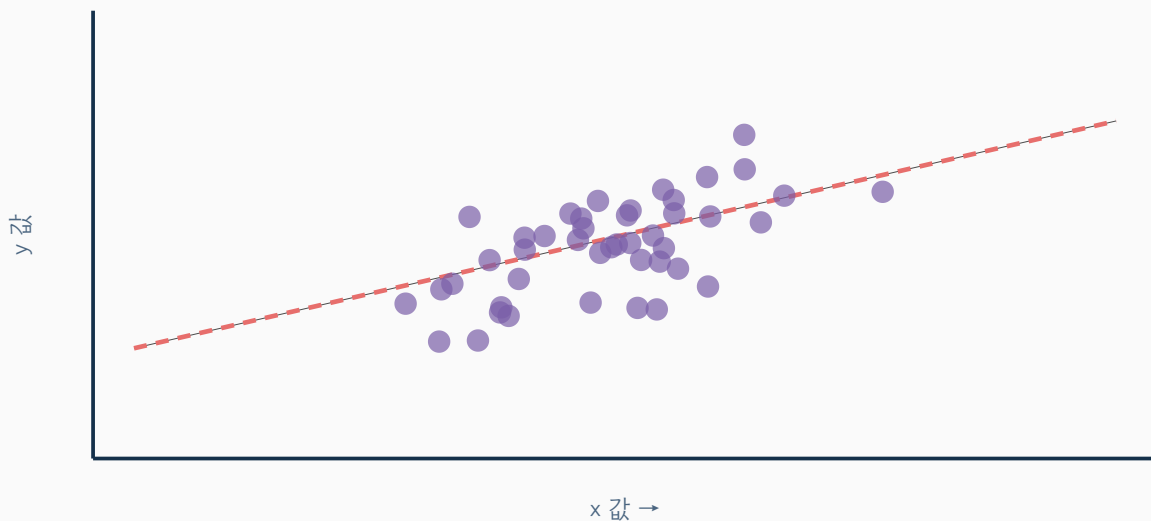
```
# bins = 막대(구간) 개수. 크게 하면 더 잘게 나뉜다
plt.hist(age, bins=20, color='skyblue', edgecolor='black')
plt.axvline(age.mean(), color='red', linestyle='dashed') # 평균 세로선
plt.show()
```

Q5 산점도 — 두 값의 관계

점 하나가 (x, y) 한 쌍 입니다. 점들이 **오른쪽 위로 몰리면** "x가 커질수록 y도 커진다"(양의 관계), 흩어져 있으면 "별 관계 없다"는 뜻이에요.

산점도 실험실

슬라이더로 두 값의 관계 세기를 바꿔보세요. 관계가 강해질수록 점들이 **한 줄(추세선)**로 모입니다.



관계의 세기

보통

상관계수 r (-1~1)

0.55

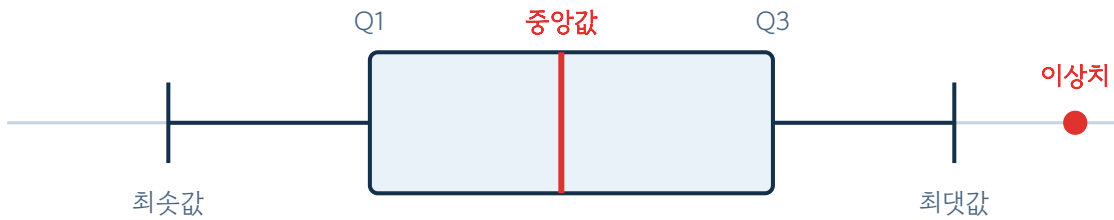
점 다시 뿌리기

살짝 우상향이 보이죠? 약~보통 정도의 관계입니다.

```
# s = 점 크기, alpha = 투명도(겹친 점 보기)
plt.scatter(ad_spend, sales, color='purple', alpha=0.6, s=50)
plt.xlabel('ad spend')    # 광고비 (x)
plt.ylabel('sales')       # 매출 (y)
plt.show()
```

Q6 박스플롯 — 요약과 이상치

데이터를 다섯 숫자로 요약합니다: 최솟값 · Q1(25%) · 중앙값(50%) · Q3(75%) · 최댓값. 상자(box)는 가운데 50%가 들어있는 구간이고, 상자 밖 멀리 떨어진 점은 이상치입니다.



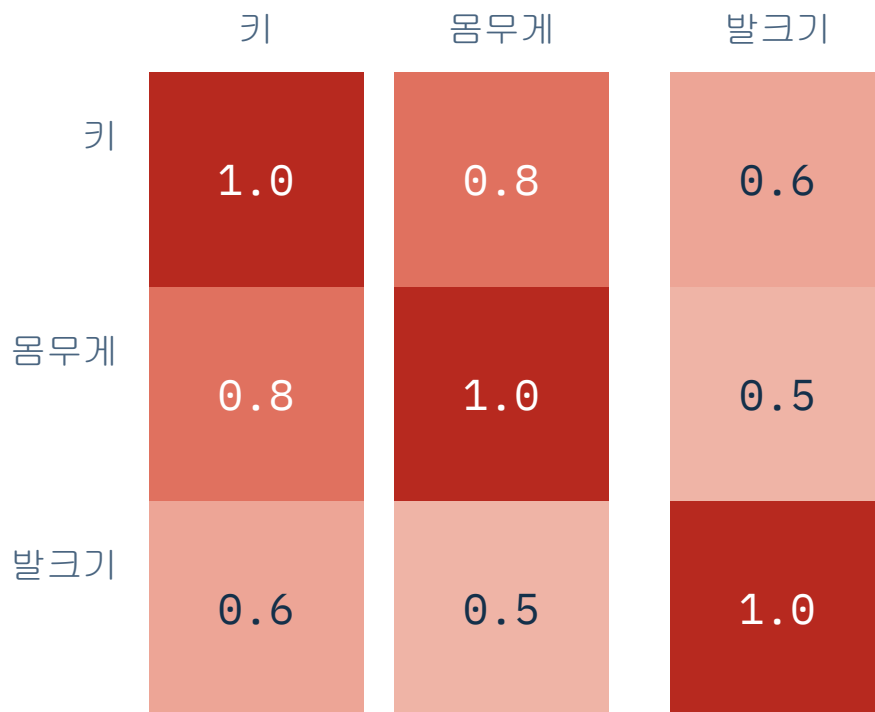
상자 = 가운데 50% · 상자 밖 먼 점 = 이상치

중앙값이 어디쯤인지, 데이터가 얼마나 퍼졌는지, 튀는 값이 있는지 한 그림에.

박스플롯이 좋은 이유 — 평균 하나만 보면 놓치는 "퍼짐"과 "튀는 값"을 함께 보여줍니다. 그룹을 나란히 그리면(요일별·반별) 어느 그룹이 더 높고 더 들쭉날쭉한지 바로 비교돼요.

Q7 여러 변수를 한눈에 — 히트맵

변수가 많아지면 관계를 하나씩 보기 힘듭니다. **히트맵**은 변수들끼리의 **상관계수(-1~1)**를 **색의 진하기**로 칠해, 어떤 변수끼리 붙어 다니는지 한눈에 보여줍니다.



색이 진할수록 관계가 강함 (1에 가까움)

키·몸무게·발 크기는 서로 강하게 붙어 다닙니다(진한 색).

페어플롯(pairplot)도 자주 씁니다. 변수들을 **둘씩 짝지어 산점도를 격자로** 짝 그려줘서, 어떤 두 변수가 잘 나뉘고 관계있는지 **한 판에** 볼 수 있어요. (seaborn의 `sns.pairplot`)

Q9 Level별 코드 작성하기

옆에 주피터 노트북을 열고, 복사 버튼으로 코드를 붙여넣어 따라 해보세요. matplotlib 기초부터 seaborn·공개 데이터까지 3단계입니다.

코드 해석 ▼을 누르면 한 줄씩 쉽게 풀어 줍니다.

LEVEL 1 단순(matplotlib) — 선·히스토그램·산점도 한 판에

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 100)
y = np.sin(x) # 시간에 따른 변화(가정)
age = np.random.normal(35, 10, 500) # 나이 분포(평균35)
ad = np.random.uniform(100, 1000, 50)
sales = ad * 2.5 + np.random.normal(0, 500, 50) # 광고비↔매출

fig, ax = plt.subplots(1, 3, figsize=(13, 4)) # 그래프 3개를 나란히
ax[0].plot(x, y, color='orange'); ax[0].set_title('Line')
ax[1].hist(age, bins=20, color='skyblue', edgecolor='black'); ax[1].set_title('Histogram')
ax[2].scatter(ad, sales, color='purple', alpha=0.6); ax[2].set_title('Scatter')
plt.tight_layout(); plt.show()
```

```
import numpy as np / matplotlib.pyplot as plt
```

숫자 배열을 만드는 numpy와 그림 도구 matplotlib을 불러옵니다. **plt**가 그래프를 그리는 담당이
예요.

```
x = np.linspace(0, 10, 100) / y = np.sin(x)
```

0~10을 100칸으로 나눈 x와, 그에 따라 오르내리는 y를 만듭니다(시간 변화 흉내).

```
age = np.random.normal(35, 10, 500)
```

평균 35, 퍼짐 10인 나이 500개를 무작위로 만듭니다. 히스토그램용 '분포' 데이터예요.

```
sales = ad * 2.5 + ...noise
```

광고비(ad)에 비례해 매출을 만들되 약간의 잡음을 더합니다 → 산점도에서 우상향 관계가 보이게.

```
fig, ax = plt.subplots(1, 3, ...)
```

한 줄(1)에 그래프 3개(3)를 나란히 놓는 판을 만듭니다. **ax[0]·ax[1]·ax[2]**가 각 칸이에요.

```
ax[0].plot / ax[1].hist / ax[2].scatter
```

각 칸에 선·히스토그램·산점도를 그립니다. **set_title** 로 칸마다 제목을 붙여요.

```
plt.tight_layout(); plt.show()
```

그래프끼리 안 겹치게 간격을 정리하고 화면에 띄웁니다.

LEVEL 2 seaborn — 내장 '팁' 데이터로 분포 + 박스플롯

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset('tips')          # 식당 팁 데이터 (seaborn 내장)

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
# 왼쪽: 결제액 분포 (히스토그램 + 밀도곡선)
sns.histplot(data=tips, x='total_bill', bins=30, kde=True, color='skyblue', ax=ax[0])
# 오른쪽: 요일별 결제액 박스플롯
sns.boxplot(data=tips, x='day', y='total_bill', hue='day', palette='pastel', ax=ax[1])
plt.tight_layout(); plt.show()
```

```
import seaborn as sns
```

seaborn은 matplotlib 위에 있어 **더 예쁘고 쉽게** 그리는 도구입니다. 보통 **sns**로 줄여 써요.

```
tips = sns.load_dataset('tips')
```

seaborn에 들어 있는 유명한 '식당 팁' 표를 불러옵니다(결제액·팁·요일·인원 등). 연습용으로 딱이에요.

```
sns.histplot(..., kde=True)
```

결제액이 어떻게 퍼졌는지 히스토그램으로 그리고, **kde=True**로 매끄러운 밀도곡선까지 엽니다.

```
sns.boxplot(x='day', y='total_bill', ...)
```

요일(day)별로 결제액(total_bill) 박스플롯을 그립니다. **요일마다 상자를 나란히** 놓아 비교돼요.

```
ax=ax[0] / ax=ax[1]
```

어느 칸에 그릴지 지정합니다. seaborn도 **ax**로 위치를 정해요.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# 공개 URL로 펭귄 데이터 바로 불러오기
url = 'https://raw.githubusercontent.com/mwaskom/seaborn-data/master/penguins.csv'
df = pd.read_csv(url).dropna()          # 빈 값 있는 행 제거
num = df.select_dtypes(include='number') # 숫자 열만 고르기
corr = num.corr()                       # 변수끼리 상관계수 표 (-1~1)

plt.figure(figsize=(6, 5))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f') # 색+숫자로 표시
plt.title('penguins correlation'); plt.tight_layout(); plt.show()
```

```
import pandas as pd / seaborn / matplotlib
```

표를 다루는 pandas, 그림 도구 seaborn·matplotlib을 준비합니다.

```
df = pd.read_csv(url).dropna()
```

펭귄 데이터를 **공개 URL에서 바로** 읽고, 빈 값이 있는 행은 버립니다(`dropna`).

```
num = df.select_dtypes(include='number')
```

상관계수는 숫자끼리만 쥔 수 있어서, **숫자 열만** 골라냅니다(종·섬 같은 글자 열 제외).

```
corr = num.corr()
```

모든 숫자 변수쌍의 **상관계수(-1~1)** 표를 만듭니다. 1에 가까울수록 함께 커지는 사이.

```
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
```

그 표를 **색(진하기)**으로 칠하고(`cmap`), **숫자**도 함께 표시합니다(`annot` , 소수 2자리). 부리 길이와 지느러미 길이가 강하게 붙어 다니는 게 한눈에 보여요.